

Operating SkyNYC

The daily routine, the credit budget, the logs, and what to do when something breaks.

Covers: supervision model · health checks · credit accounting · log reading · failure playbook · recovery | August 2026

1. How the system stays alive — and who restarts what

Before you operate anything, know who is responsible for keeping it running. In SkyNYC there is exactly one supervisor for the streaming side, and it is Docker. Every long-running service — both producers and the weather consumer — carries a restart policy, so a crashed process comes back on its own within seconds. Later milestones add a batch scheduler for daily and hourly jobs, but it will never supervise these streams: schedulers are built for jobs that finish, and a stream never finishes. Keep that division of labor straight and half the confusing questions disappear.

The production home for all of this is the small always-on server from guide 3 — every check in this guide runs there over SSH exactly as written. If you ever collect on a laptop instead (a demo, a dev experiment), remember it has no supervisor: a machine that sleeps takes the pipeline with it, and the hole it leaves is permanent — the aircraft API serves at most one hour of history, so there is no going back to refill a gap. The compose file is the portability story; nothing changes but the host.

2. The morning routine

Four checks, in order, about ninety seconds total. Run them at the start of any day the pipeline is supposed to be collecting — and always before you change anything.

#	CHECK	COMMAND	HEALTHY LOOKS LIKE
1	Processes	<code>docker compose ps</code>	All services <i>Up</i> ; Kafka and Postgres (<i>healthy</i>); nothing flapping
2	Credit budget	<code>make credits</code>	Comfortably above 800 remaining by mid-day
3	Data freshness	<code>make psql</code> then the two queries below	Weather under 90 minutes per station
4	Consumer lag	<code>make lag</code>	Lag at 0 or single digits on both weather topics
5	Batch layer	Dagster UI over the tunnel (port 3001)	Both schedules show a recent tick; last runs green. Schedules ship off — if a schedule shows no ticks ever, it was never toggled on

```
-- freshness, per station (should be < 90 min; observations are hourly by nature)
SELECT station, now() - max(obs_ts) AS age FROM weather_obs GROUP BY station;

-- yesterday's alert activity, a quick sanity read
SELECT airport, event, severity, effective_ts, ends_ts
FROM weather_alerts ORDER BY effective_ts DESC LIMIT 10;
```

WHY LAG ONLY SHOWS THE WEATHER CONSUMER

`make lag` lists consumer groups known to Kafka, and only the weather consumer registers one. The stream-processing queries track their own positions internally and *never appear here*. Judge the streaming side by data freshness and the dashboard's freshness stat, not by this listing — an empty lag report for it is normal, not a problem.

3. The credit budget — the one number to respect

OpenSky grants 4,000 request credits per day, and the states poll spends one per call. At the normal 30-second cadence that is 2,880 per day, leaving roughly 28% headroom for restarts, debugging, and your own curiosity. The producer manages this budget for you in two ways:

- It logs the remaining balance on every single poll — that is the `credits_remaining=...` field in its log line, and it is what `make credits` reads.
- If the balance ever drops below 500, it automatically slows itself to 60-second polling and says so in the log. If you see that warning, something has been spending credits outside the normal cadence — find out what before overriding anything.

YOUR EXPERIMENTS SPEND FROM THE SAME WALLET

A manual states request in a terminal costs exactly what the producer's poll costs. One-off checks are fine; a shell loop is not. If the budget is ever exhausted, the API answers 429 and tells the producer how long to wait — it sleeps precisely that long and resumes. Nothing for you to do but identify what burned the budget.

4. Reading the logs

Each service logs one meaningful line per unit of work. Learn these three shapes and you can read the whole pipeline at a glance with `make logs`.

The aircraft producer

```
poll ok n_states=223 published=223 null_pos=0 parse_drops=0 credits_remaining=3996 interval=30s
```

FIELD	MEANING	WORTH A LOOK WHEN...
<code>n_states</code>	Aircraft in the box this poll	0 outside the 03:00–05:00 lull
<code>null_pos</code>	Rows with no position — counted, still published; dropped later at the processing layer	Suddenly a large share of <code>n_states</code>
<code>parse_drops</code>	Rows that failed to parse at all	Anything persistently non-zero — the upstream shape changed
<code>credits_remaining</code>	The budget, live	Below ~1,100 before noon

The weather producer

```
KJFK obs published obs_ts=... category=VFR wind_kmh=18.5 vis_m=16093 ceiling_ft=None  
KEWR obs unchanged at ... — dedupe skip
```

Both lines are healthy. Underlying observations update roughly hourly — the 5-minute poll exists to catch the special reports that stations issue when conditions change quickly. So most polls are deduplicate-skips, and that is the system working, not idling. Empty `wind gust` and `ceiling` fields are normal too: gusts are only reported when gusting, and a ceiling only exists under a broken or overcast sky. All units are converted at parse time by reading each field's declared unit code — never assumed from the field name — because the upstream mixes km/h, meters, and degrees Celsius in one payload.

The weather consumer

```
upserted wx.observations key=KJFK
```

Every write is an upsert on the natural key, and its offset is committed only after the row lands. A crash between write and commit replays the message into the same row — you cannot duplicate data by restarting this service, so never hesitate to restart it.

5. Spot-checking the stream

When you want to see the actual messages rather than trust the logs:

```
# two live aircraft messages, straight off the topic
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic adsb.states.raw \
  --max-messages 2 --timeout-ms 45000

# message counts per partition — should grow by a few hundred every 30 s
docker compose exec kafka /opt/kafka/bin/kafka-run-class.sh \
  org.apache.kafka.tools.GetOffsetShell \
  --bootstrap-server localhost:9092 --topic adsb.states.raw
```

Expect a compact JSON envelope per aircraft: transponder address, callsign, position, altitude, speed, track, vertical rate, an on-ground flag, and two timestamps. Null altitude on an aircraft that is on the ground is normal and verified against the live source — the processing layer treats nulls as missing samples, not errors.

6. The failure playbook

Work top to bottom; stop at the first row that matches. The golden rule: identify before you restart — a restart destroys the evidence and usually was not the fix anyway.

SYMPTOM	WHAT IT MEANS	WHAT TO DO
Container restarting in a loop	Nine times out of ten, a missing or malformed <code>.env</code> value	<code>docker compose logs <service></code> — read the <i>first</i> traceback
Repeated 401 in the aircraft producer log	Token refresh is failing — credentials are wrong, not expired	Re-check both OpenSky values in <code>.env</code> ; rotate the client if in doubt
429 and a long sleep in the log	Daily credits exhausted	Nothing to fix right now — it resumes by itself. Investigate what overspent: scan the day's <code>credits_remaining</code> trend
403 in the weather producer log	Missing or blank contact email in the User-Agent	Set <code>CONTACT_EMAIL</code> in <code>.env</code> and restart the producer
Backoff lines climbing (30 s → 60 s → ... → 8 min)	Upstream 5xx or timeouts — their incident, not yours	Wait. The producer retries with increasing patience and resumes alone; a data gap is honest and unavoidable
A station's observation older than 2 h	Station gap upstream (it happens) or your producer is down	Check the producer log first; if it is polling and skipping, the station itself is quiet — note it and move on
Consumer lag climbing steadily	Postgres unreachable or slow; the consumer retries the same message until it lands	Check Postgres health, then the consumer log — it says exactly what it is waiting for
Wind values roughly double what the forecast says	A unit regression — something read km/h as m/s	That is a code defect, not an ops issue. File it; do not hand-correct data

7. Recovery semantics — what a restart actually does

Everything in this pipeline is built to be killed. Knowing what each restart does makes you appropriately fearless:

- **Aircraft producer:** on start it fetches a fresh token and resumes polling. The moments it was down are simply polls that never happened — a legitimate gap, recorded as absence, never fabricated after the fact.
- **Weather producer:** resumes polling; the first observation per station may republish once, and the consumer's upsert makes that invisible downstream.
- **Weather consumer:** resumes from its last committed offset; at-least-once delivery plus idempotent writes means restarts are always safe.
- **Kafka / Postgres:** data lives in named volumes and survives any restart. `make down` keeps volumes. The `-v` variant destroys them — that is an explicit, deliberate decision (see below), never a reflex.

THE TWO THINGS YOU MUST NEVER DO CASUALLY

1. `docker compose down -v` — the Kafka volume *is* the raw history, and the upstream API cannot re-serve it. Deleting it is deleting the only archive.
2. Manual states-request loops — they spend the same credit budget the pipeline lives on. Both are recoverable never; ask twice, type once.

8. Stopping and starting

INTENT	COMMAND	EFFECT
Pause everything, keep all data	<code>make down</code>	Containers stop; volumes, topics, and rows survive
Resume	<code>make up</code> then <code>make ingest</code>	Producers pick up where the world is now; the consumer picks up where it left off
Bounce one service	<code>docker compose restart <service></code>	Safe at any time, for any service in this stack
Follow one service's behavior	<code>docker compose logs -f <service></code>	Live tail; add <code>--tail 50</code> for recent context

After any stop of more than a few minutes, run the morning routine from section 2 before trusting the data again. It takes ninety seconds and it has caught every silent misconfiguration this pipeline has ever had.